

Complete RAG (Retrieval-Augmented Generation) Guide

Table of Contents

1. Introduction to RAG
 2. Why RAG is Needed
 3. RAG Architecture
 4. What is Chunking?
 5. Chunking Strategies
 - Fixed Size Chunking
 - Recursive Character Splitter
 - Document Based Chunking
 - Token Based Chunking
 - Semantic Chunking
 6. What are Embeddings?
 7. What is a Vector Database?
 8. What is a Retriever?
 9. Similarity Search
 10. Maximal Marginal Relevance (MMR)
 11. Re-Ranking
 12. Final End-to-End RAG Flow
 13. Production-Level RAG Insights
-

1. Introduction to RAG

What is RAG?

RAG stands for Retrieval-Augmented Generation.

It is an AI architecture that combines:

1. Retrieval System
2. Large Language Model (LLM)

Instead of depending only on the knowledge stored inside the LLM, RAG retrieves external information dynamically and gives it to the model before generating an answer.

Technical Definition

Retrieval-Augmented Generation is a framework where relevant documents are retrieved from an external knowledge source and injected into the prompt context of a Large Language Model to improve factual accuracy and reduce hallucinations.

Simple Explanation

Traditional LLM:

Question → Model → Answer

RAG:

Question → Retrieve Relevant Data → LLM → Better Answer

Why RAG Became Important

LLMs have limitations:

- Knowledge cutoff
- Hallucinations
- No access to private company data
- Expensive retraining

RAG solves these problems by giving external context.

Real-World Example

Imagine a company chatbot.

Without RAG:

- ChatGPT may not know company leave policy.

With RAG:

- System retrieves leave policy document.
 - LLM reads retrieved content.
 - Generates accurate answer.
-

Example

User Question:

"How many annual leaves do employees get?"

Retrieved Context:

"Employees are eligible for 18 annual leaves per year."

LLM Answer:

"Employees get 18 annual leaves per year."

2. Why RAG is Needed

Problems with Traditional LLMs

1. Hallucination

LLMs sometimes generate incorrect information confidently.

Example:

Question: "What is my company leave policy?"

Without data:

- Model may generate fake policy.
-

2. Knowledge Cutoff

LLMs are trained on old data.

They may not know:

- Latest news
 - Recent company updates
 - Newly added documents
-

3. No Private Knowledge

Companies have internal data:

- HR policies
- Technical documentation
- Internal PDFs
- Customer records

LLMs do not know this information by default.

4. Expensive Fine-Tuning

Retraining models frequently is expensive.

RAG avoids retraining.

Instead:

- Add documents
 - Retrieve relevant content dynamically
-

How RAG Solves These Problems

RAG Pipeline

1. Store documents
 2. Convert to embeddings
 3. Store in vector database
 4. Retrieve relevant chunks
 5. Send context to LLM
 6. Generate answer
-

Benefits of RAG

- Reduces hallucination
 - Access to latest data
 - Works with private documents
 - No need to retrain LLM
 - More accurate responses
 - Scalable architecture
-

3. RAG Architecture

End-to-End Architecture

User Query ↓ Embedding Model ↓ Vector Database ↓ Retriever ↓ Relevant Chunks ↓ LLM ↓ Final Answer

Step-by-Step Explanation

Step 1 — Document Loading

Documents can come from:

- PDFs
 - Databases
 - APIs
 - Text files
 - Websites
-

Step 2 — Chunking

Large documents are split into smaller chunks.

Why?

- LLM context window limitations
 - Better retrieval granularity
-

Step 3 — Embedding Generation

Each chunk is converted into numerical vectors.

Example:

"Employees get 18 leaves" → [0.21, 0.88, 0.44, ...]

Step 4 — Store in Vector DB

Embeddings are stored in vector databases.

Examples:

- FAISS
 - Chroma
 - Pinecone
 - Weaviate
-

Step 5 — Query Processing

User question is converted into embedding.

Step 6 — Retrieval

Retriever finds most similar chunks.

Step 7 — Context Injection

Retrieved chunks are added to prompt.

Step 8 — LLM Generation

LLM generates final response using retrieved context.

4. What is Chunking?

Definition

Chunking is the process of splitting large documents into smaller manageable pieces called chunks.

Why Chunking is Important

Without chunking:

- Large documents exceed token limits
 - Retrieval becomes inefficient
 - Irrelevant information increases
-

Benefits of Chunking

- Better retrieval accuracy
 - Better semantic understanding
 - Reduced token usage
 - Faster search
-

Example

Original Document:

"Employees are eligible for 18 annual leaves per year. Leave approval requires manager permission. Salary is credited monthly."

Chunks:

Chunk 1: "Employees are eligible for 18 annual leaves per year."

Chunk 2: "Leave approval requires manager permission."

Chunk 3: "Salary is credited monthly."

5. Chunking Strategies

5.1 Fixed Size Chunking

Definition

Fixed-size chunking splits text using constant size.

Can be based on:

- Characters
- Words

- Tokens
-

Example

Text:

"Employees are eligible for 18 annual leaves per year. Leave approval requires manager permission."

Chunk Size = 50 characters

Output:

Chunk 1: "Employees are eligible for 18 annual leaves"

Chunk 2: "per year. Leave approval requires manager"

Chunk 3: "permission"

Advantages

- Simple
 - Fast
 - Easy implementation
-

Disadvantages

- Breaks sentences
 - Loses semantic meaning
 - Poor retrieval quality
-

Use Cases

- Small systems
 - Baseline implementations
-

5.2 Recursive Character Splitter

Definition

Recursive Character Splitter splits text hierarchically using multiple separators.

It tries:

1. Paragraph
 2. Line
 3. Sentence
 4. Word
 5. Character
-

Why Recursive?

If chunk remains too large:

- It recursively tries smaller separators.
-

Separator Order

["\n\n", "\n", " ", ""]

Example

Input:

Employees are eligible for 18 annual leaves.

Leave approval requires manager permission.

Salary is credited monthly.

Output:

Chunk 1: Employees are eligible for 18 annual leaves.

Chunk 2: Leave approval requires manager permission.

Chunk 3: Salary is credited monthly.

Advantages

- Preserves meaning
 - Better semantic boundaries
 - Production-ready
-

Disadvantages

- Slightly slower
 - More complex
-

Use Cases

- Production RAG systems
 - Chatbots
 - Enterprise search
-

5.3 Document-Based Chunking

Definition

Document-based chunking splits documents based on structure.

Examples:

- Headings
 - Sections
 - Chapters
 - Titles
-

Example

Document:

Leave Policy

Employees get 18 annual leaves.

Salary

Salary credited monthly.

Chunks:

Chunk 1: Leave Policy → Employees get 18 annual leaves.

Chunk 2: Salary → Salary credited monthly.

Advantages

- Preserves logical structure
 - High-quality retrieval
 - Context aware
-

Disadvantages

- Requires structured documents
 - Hard for unstructured text
-

Use Cases

- PDFs
 - Technical documentation
 - HR policies
 - Legal documents
-

5.4 Token-Based Chunking

Definition

Token-based chunking splits text based on token count.

Important because: LLMs process tokens, not words.

Example

Sentence:

"ChatGPT is powerful"

Possible tokens:

["Chat", "G", "PT", " is", " powerful"]

Why Important?

LLMs have token limits.

Example:

- GPT-4 context window
- Llama token limits

Token chunking prevents overflow.

Advantages

- Model-aware
 - Efficient token usage
 - Prevents truncation
-

Disadvantages

- Requires tokenizer
 - Less human readable
-

Use Cases

- Production systems
 - LLM optimization
 - Large-scale RAG
-

5.5 Semantic Chunking

Definition

Semantic chunking splits text based on meaning instead of size.

Goal:

- Keep related ideas together
-

Example

Text:

Employees get 18 annual leaves. Leave approval requires permission. Salary credited monthly.

Output:

Chunk 1: Employees get 18 annual leaves. Leave approval requires permission.

Chunk 2: Salary credited monthly.

Advantages

- Best retrieval quality

- Context preservation
 - High semantic understanding
-

Disadvantages

- Complex implementation
 - Computationally expensive
-

Use Cases

- Advanced RAG systems
 - Semantic search
 - Enterprise AI
-

6. What are Embeddings?

Definition

Embeddings are dense numerical vector representations of text that capture semantic meaning.

Simple Explanation

Embeddings convert text into numbers.

Similar meaning → similar vectors.

Example

Dog → [0.21, 0.88, 0.11] Cat → [0.20, 0.85, 0.15] Car → [0.90, 0.11, 0.33]

Dog and Cat are close because meanings are similar.

Why Embeddings are Important

Computers cannot understand raw text.

They understand numerical vectors.

Embeddings allow:

- Semantic search
 - Similarity comparison
 - Retrieval systems
-

Types of Embeddings

Word Embeddings

- Word2Vec
- GloVe

Sentence Embeddings

- Sentence Transformers

Document Embeddings

- Full document vectors
-

Real Example in RAG

Stored Chunks:

Chunk 1 → Employees get 18 leaves Chunk 2 → Salary credited monthly

User Query:

"How many leaves do employees get?"

Embedding similarity retrieves Chunk 1.

7. What is a Vector Database?

Definition

A vector database stores embeddings and performs efficient similarity search.

Why Vector DB Needed

Normal databases are not optimized for:

- Vector similarity
 - High-dimensional search
 - Semantic retrieval
-

What Vector DB Stores

1. Embeddings
2. Metadata

Example:

Vector: [0.21, 0.44, 0.66]

Metadata: { "source": "HR.pdf", "page": 3 }

How It Works

Query → Embedding → Similarity Search → Top Results

Similarity Metrics

Cosine Similarity

Most common.

Measures angle between vectors.

Euclidean Distance

Measures distance between vectors.

Popular Vector Databases

- FAISS
 - Chroma
 - Pinecone
 - Weaviate
 - Milvus
-

Real Example

Stored:

Chunk 1 → Leave policy Chunk 2 → Salary policy

Query: "Leave rules"

Vector DB retrieves Chunk 1.

8. What is a Retriever?

Definition

Retriever is a component that fetches the most relevant chunks from the vector database.

Purpose

Retriever decides:

"Which data should the LLM see?"

RAG Flow

Query ↓ Embedding ↓ Retriever ↓ Top Relevant Chunks ↓ LLM

Example

Stored Chunks:

1. Employees get 18 leaves
2. Salary credited monthly
3. Office timing 9–5

Query: "Leave policy"

Retriever Output: Chunk 1

Types of Retrievers

Dense Retriever

Uses embeddings.

Sparse Retriever

Uses keyword matching.

Hybrid Retriever

Combines dense + sparse retrieval.

Importance of Retriever

Bad retriever:

- Wrong chunks
- Wrong answers

Good retriever:

- Better context
 - Better answers
-

9. Similarity Search

Definition

Similarity search finds vectors that are closest to query vector.

How It Works

1. Query converted to embedding
 2. Compare with stored embeddings
 3. Return top similar vectors
-

Example

Query: "How many leaves do employees get?"

Stored Chunks:

Chunk 1 → Employees get 18 annual leaves
Chunk 2 → Salary credited monthly
Chunk 3 → Office timing 9–5

Similarity Scores

Chunk 1 → 0.97 Chunk 2 → 0.22 Chunk 3 → 0.10

Output

Chunk 1 retrieved.

Common Similarity Algorithms

Cosine Similarity

Measures vector angle.

Dot Product

Measures vector alignment.

Euclidean Distance

Measures physical distance.

10. Maximal Marginal Relevance (MMR)

Definition

MMR retrieves documents that are:

1. Relevant to query
 2. Different from each other
-

Problem with Normal Retrieval

Retriever may return duplicate information.

Example:

1. Employees get 18 leaves
2. Leave policy allows 18 days
3. Annual leave count is 18

All almost same meaning.

How MMR Solves This

MMR balances:

Relevance + Diversity

Example

Query: "Explain leave policy"

Normal Retriever Output:

1. Leave count
 2. Leave count duplicate
 3. Leave count duplicate
-

MMR Output:

1. Leave count
 2. Sick leave policy
 3. Leave approval process
-

MMR Formula

MMR = Relevance – Similarity with already selected chunks

Benefits

- Reduces duplication
 - Better context diversity
 - More informative retrieval
-

Use Cases

- Large document systems
 - Enterprise RAG
 - Conversational AI
-

11. Re-Ranking

Definition

Re-ranking reorders retrieved documents using a more accurate model.

Why Re-Ranking Needed

Retriever is fast but not perfectly accurate.

Re-ranker improves final ordering.

Retrieval Pipeline

Query ↓ Retriever ↓ Top K Chunks ↓ Re-Ranker ↓ Best Ordered Chunks ↓ LLM

Example

User Query:

"How many leaves do employees get?"

Retriever Output:

1. Leave approval process
 2. Employees get 18 leaves
 3. Salary policy
-

Problem: Correct answer not first.

Re-Ranker Scores

Chunk 2 → 0.99 Chunk 1 → 0.82 Chunk 3 → 0.10

Final Re-Ranked Output

1. Employees get 18 leaves

2. Leave approval process
 3. Salary policy
-

Cross Encoder Models

Most re-rankers use cross encoders.

Input: (Query + Document) → Relevance Score

Benefits

- Better relevance
 - Higher answer quality
 - Reduced hallucinations
-

Trade-Off

More accurate but slower.

12. Final End-to-End RAG Flow

Complete Flow

Documents ↓ Chunking ↓ Embeddings ↓ Vector Database ↓ Retriever ↓ MMR ↓ Re-Ranking
↓ LLM ↓ Final Answer

Example End-to-End Scenario

Documents:

1. Employees get 18 leaves
2. Salary credited monthly
3. Sick leave separate

User Query:

"How many leaves do employees get?"

Pipeline:

1. Query converted to embedding
 2. Similarity search performed
 3. Retriever fetches chunks
 4. MMR removes duplicates
 5. Re-ranker improves order
 6. LLM generates final answer
-

Final Answer:

"Employees get 18 annual leaves per year."

13. Production-Level RAG Insights

Important Engineering Concepts

Chunk Quality Matters

Bad chunking → poor retrieval.

Embedding Quality Matters

Better embeddings → better semantic understanding.

Retriever Quality Matters

Retriever decides what context reaches the LLM.

Re-Ranking Improves Precision

Retriever gives candidates. Re-ranker finds best results.

Typical Production Pipeline

Recursive Chunking + Embeddings + Vector Database + Hybrid Retrieval + MMR + Re-Ranking + LLM

Final Summary

RAG is one of the most important architectures in modern Generative AI.

It enables:

- Accurate AI systems
- Dynamic knowledge retrieval
- Reduced hallucination
- Enterprise AI applications

Core Components:

1. Chunking
2. Embeddings
3. Vector Database
4. Retriever
5. Similarity Search
6. MMR
7. Re-Ranking

Together, these components build scalable, production-ready AI systems.